

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 14.

«РАЗРАБОТКА СЕРВИСНОЙ ЧАСТИ ПРОГРАММЫ».

Выполнил: Рахманов А. И.

Студент 24-ИС.

2026г.

Введение

Цель

Изучить принципы построения сервисного слоя (Service Layer) в приложении, освоить паттерны **Repository**, **Service**, **Dependency Injection**, научиться проектировать бизнес-логику, отделённую от инфраструктурного кода, и покрывать её юнит-тестами.

Базовое задание (образец выполнения)

Общее условие (для всех вариантов)

Разработать сервисную часть для **системы управления библиотекой**.

Бизнес-требования:

- Хранить книги (название, автор, ISBN, год, жанр, количество экземпляров).
- Хранить читателей (имя, email, телефон).
- Осуществлять выдачу книг читателям (с проверкой наличия экземпляров и задолженностей).
- Возврат книг (увеличение доступных экземпляров, расчёт штрафа при просрочке).
- Поиск книг по автору, названию, жанру.

Вариант

Таблица 1. Варианты с предметной областью и требованиями

	Предметная область	Особенности бизнес-логики	Дополнительные слои
5	Управление задачами (Trello-like)	Доски, колонки, теги, назначение исполнителей, сроки	История изменений (аудит)

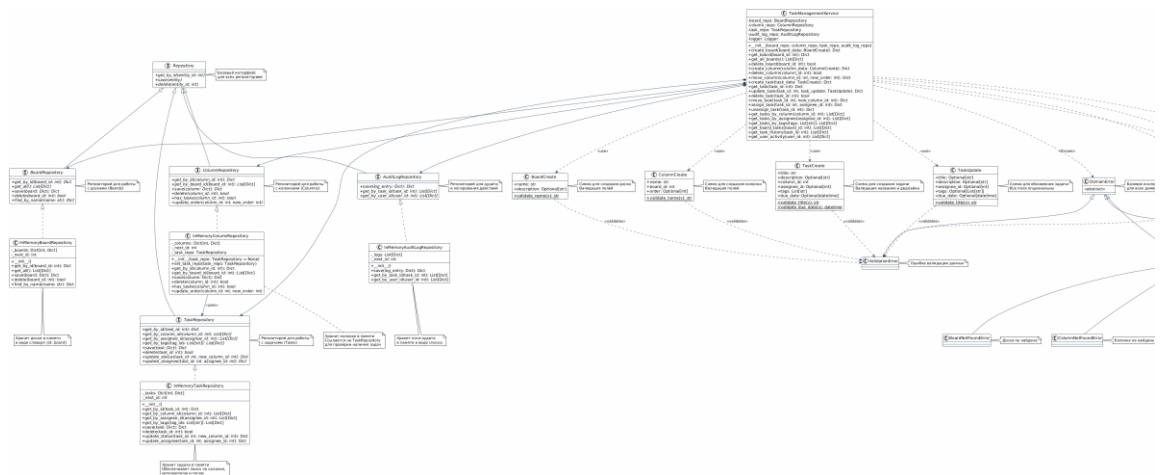
Структура интерфейса

Day14

```
|— src/
|   |— repositories/
|       |— __init__.py
|       |— in_memory.py
|       |— interfaces.py
```

- | |— services/
 - | |— __init__.py
 - | |— library_service.py
 - | |— task_service.py
- | |— __init__.py
- | |— exceptions.py
- |— tests/
 - | |— test_service.py
 - | |— test_library_service.py
- |— schemas.py
- |— problem_code.py

Диаграмма зависимостей



Приложение 1. Диаграмма зависимостей

Анализ проблемного кода

Код до анализа

```
import datetime
class LibraryService:
    # Жёсткая связанность: сервис сам создаёт структуры данных
    def __init__(self):
        self.books = [] # список словарей (нарушение инкапсуляции)
        self.readers = []
        self.loans = []

    def add_book(self, title, author, isbn, year, genre, copies):
        # Нет валидации: copies может быть отрицательным
        # Нет проверки на отрицательное количество
        # Нет проверки на пустые строки
        # Нет проверки на дубликат ISBN
```

```

        self.books.append({
            "id": len(self.books) + 1,
            "title": title,
            "author": author,
            "isbn": isbn,
            "year": year,
            "genre": genre,
            "copies": copies
        })

    def register_reader(self, name, email, phone):
        self.readers.append({
            "id": len(self.readers) + 1,
            "name": name,
            "email": email,
            "phone": phone
        })

    def lend_book(self, book_id, reader_id):
        # Эти две операции должны быть в одной транзакции
        # Нет проверки на долги читателя
        # Нет проверки на уже выданную книгу
        # Нет проверки существования книги и читателя
        book = next((b for b in self.books if b["id"] == book_id), None)
        if not book:
            return "Книга не найдена"
        if book["copies"] <= 0:
            return "Нет экземпляров"
        reader = next((r for r in self.readers if r["id"] == reader_id),
None)

        if not reader:
            return "Читатель не найден"
        # Уменьшаем количество
        book["copies"] -= 1
        # Запись о выдаче (без проверки долгов)
        self.loans.append({
            # Если между ними произойдёт ошибка - данные несогласованы
            "book_id": book_id,
            "reader_id": reader_id,
            "loan_date": datetime.date.today(),
            "return_date": None
        })
        return "Книга выдана"

    def return_book(self, book_id, reader_id):
        loan = next((l for l in self.loans if l["book_id"] == book_id and
l["reader_id"] == reader_id and l["return_date"] is
None), None)

        if not loan:
            return "Выдача не найдена"
        loan["return_date"] = datetime.date.today()
        book = next((b for b in self.books if b["id"] == book_id), None)
        if book:

```

```
        book["copies"] += 1
    # Штраф не считается
    return "Книга возвращена"
```

Приложение 2. problem_code

Код после анализа(разделён на несколько программ)

```
# schemas.py
from pydantic import BaseModel, Field, validator
from typing import Optional
from datetime import date

class BookCreate(BaseModel):
    """Схема создания книги"""
    title: str = Field(..., min_length=1, max_length=200)
    author: str = Field(..., min_length=1, max_length=100)
    isbn: str = Field(..., min_length=10, max_length=20)
    year: int = Field(..., ge=1450, le=date.today().year)
    genre: Optional[str] = Field(None, max_length=50)
    copies: int = Field(..., ge=0, le=1000)

    @validator('title')
    def validate_title(cls, v):
        if not v or not v.strip():
            raise ValueError('Название книги не может быть пустым')
        return v.strip()

    @validator('isbn')
    def validate_isbn(cls, v):
        # Простая проверка ISBN (упрощённо)
        if not v or len(v.replace('-', '')) < 10:
            raise ValueError('Неверный формат ISBN')
        return v.strip()

class ReaderCreate(BaseModel):
    """Схема создания читателя"""
    name: str = Field(..., min_length=1, max_length=100)
    email: str = Field(..., min_length=5, max_length=100)
    phone: str = Field(..., min_length=5, max_length=20)

    @validator('email')
    def validate_email(cls, v):
        if '@' not in v or '.' not in v:
            raise ValueError('Неверный формат email')
        return v.lower().strip()

    @validator('name')
    def validate_name(cls, v):
        if not v or not v.strip():
            raise ValueError('Имя не может быть пустым')
        return v.strip()

class LoanCreate(BaseModel):
    """Схема создания выдачи"""
    book_id: int = Field(..., gt=0)
    reader_id: int = Field(..., gt=0)
    loan_date: Optional[date] = date.today()
```

Приложение 3. schemas

```
# src/exceptions.py
class DomainError(Exception):
    """Базовое исключение для доменных ошибок"""
    pass
class ValidationError(DomainError):
    """Ошибка валидации данных"""
    pass
class NotFoundError(DomainError):
    """Сущность не найдена"""
    pass
class BusinessRuleViolation(DomainError):
    """Нарушение бизнес-правила"""
    pass
class BoardNotFoundError(NotFoundError):
    """Доска не найдена"""
    pass
class ColumnNotFoundError(NotFoundError):
    """Колонка не найдена"""
    pass
class TaskNotFoundError(NotFoundError):
    """Задача не найдена"""
    pass
class ColumnNotEmptyError(BusinessRuleViolation):
    """Колонка не пуста (содержит задачи)"""
    pass
class TaskAlreadyAssignedError(BusinessRuleViolation):
    """Задача уже назначена исполнителю"""
    pass
```

Приложение 4. exceptions

```
# repositories/in_memory.py
from typing import List, Optional, Dict
from datetime import datetime
from .interfaces import (
    BoardRepository, ColumnRepository,
    TaskRepository, AuditLogRepository
)
class InMemoryBoardRepository(BoardRepository):
    """In-Memory реализация репозитория досок"""
    def __init__(self):
        self._boards: Dict[int, Dict] = {}
        self._next_id = 1
    def get_by_id(self, board_id: int) -> Optional[Dict]:
        return self._boards.get(board_id)
    def get_all(self) -> List[Dict]:
        return list(self._boards.values())
    def save(self, board: Dict) -> Dict:
        if 'id' not in board or board['id'] is None:
            board['id'] = self._next_id
```

```

        self._next_id += 1
        board['created_at'] = board.get('created_at',
datetime.now().isoformat())
        board['updated_at'] = datetime.now().isoformat()
        self._boards[board['id']] = board
        return board
def delete(self, board_id: int) -> bool:
    if board_id in self._boards:
        del self._boards[board_id]
        return True
    return False
def find_by_name(self, name: str) -> Optional[Dict]:
    for board in self._boards.values():
        if board.get('name') == name:
            return board
    return None
class InMemoryColumnRepository(ColumnRepository):
    """In-Memory реализация репозитория колонок"""
    def __init__(self):
        self._columns: Dict[int, Dict] = {}
        self._next_id = 1
    def get_by_id(self, column_id: int) -> Optional[Dict]:
        return self._columns.get(column_id)
    def get_by_board_id(self, board_id: int) -> List[Dict]:
        return [col for col in self._columns.values()
            if col.get('board_id') == board_id]
    def save(self, column: Dict) -> Dict:
        if 'id' not in column or column['id'] is None:
            column['id'] = self._next_id
            self._next_id += 1
        column['created_at'] = column.get('created_at',
datetime.now().isoformat())
        column['updated_at'] = datetime.now().isoformat()
        self._columns[column['id']] = column
        return column
    def delete(self, column_id: int) -> bool:
        if column_id in self._columns:
            # Проверяем, есть ли задачи в колонке
            if self.has_tasks(column_id):
                return False
            del self._columns[column_id]
            return True
        return False
    def has_tasks(self, column_id: int) -> bool:
        # Этот метод будет проверять через TaskRepository
        # В реальной реализации здесь может быть связь с TaskRepository
        return False
    def update_order(self, column_id: int, new_order: int):
        if column_id in self._columns:
            self._columns[column_id]['order'] = new_order

```

```

        self._columns[column_id]['updated_at'] =
datetime.now().isoformat()
class InMemoryTaskRepository(TaskRepository):
    """In-Memory реализация репозитория задач"""
    def __init__(self):
        self._tasks: Dict[int, Dict] = {}
        self._next_id = 1
    def get_by_id(self, task_id: int) -> Optional[Dict]:
        return self._tasks.get(task_id)
    def get_by_column_id(self, column_id: int) -> List[Dict]:
        return [task for task in self._tasks.values()
                if task.get('column_id') == column_id]
    def get_by_assignee_id(self, assignee_id: int) -> List[Dict]:
        return [task for task in self._tasks.values()
                if task.get('assignee_id') == assignee_id]
    def get_by_tags(self, tag_ids: List[int]) -> List[Dict]:
        return [task for task in self._tasks.values()
                if any(tag in task.get('tags', []) for tag in tag_ids)]
    def save(self, task: Dict) -> Dict:
        if 'id' not in task or task['id'] is None:
            task['id'] = self._next_id
            self._next_id += 1
        task['created_at'] = task.get('created_at',
datetime.now().isoformat())
        task['updated_at'] = datetime.now().isoformat()
        self._tasks[task['id']] = task
        return task
    def delete(self, task_id: int) -> bool:
        if task_id in self._tasks:
            del self._tasks[task_id]
            return True
        return False
    def update_status(self, task_id: int, new_column_id: int) -> Dict:
        if task_id in self._tasks:
            self._tasks[task_id]['column_id'] = new_column_id
            self._tasks[task_id]['updated_at'] = datetime.now().isoformat()
            return self._tasks[task_id]
        return None
    def update_assignee(self, task_id: int, assignee_id: Optional[int]) ->
Dict:
        if task_id in self._tasks:
            self._tasks[task_id]['assignee_id'] = assignee_id
            self._tasks[task_id]['updated_at'] = datetime.now().isoformat()
            return self._tasks[task_id]
        return None
class InMemoryAuditLogRepository(AuditLogRepository):
    """In-Memory реализация репозитория аудита"""
    def __init__(self):
        self._logs: List[Dict] = []
        self._next_id = 1
    def save(self, log_entry: Dict) -> Dict:

```



```

        log_entry['id'] = self._next_id
        self._next_id += 1
        log_entry['timestamp'] = log_entry.get('timestamp',
datetime.now().isoformat())
        self._logs.append(log_entry)
        return log_entry
    def get_by_task_id(self, task_id: int) -> List[Dict]:
        return [log for log in self._logs
                if log.get('task_id') == task_id]
    def get_by_user_id(self, user_id: int) -> List[Dict]:
        return [log for log in self._logs
                if log.get('user_id') == user_id]

```

Приложение 5. in_memory

```

# repositories/interfaces.py
from abc import ABC, abstractmethod
from typing import List, Optional, Dict, Any
from datetime import datetime
class Repository(ABC):
    """Базовый интерфейс репозитория"""
    @abstractmethod
    def get_by_id(self, entity_id: int):
        pass
    @abstractmethod
    def save(self, entity):
        pass
    @abstractmethod
    def delete(self, entity_id: int):
        pass
class BoardRepository(ABC):
    """Репозиторий для досок"""
    @abstractmethod
    def get_by_id(self, board_id: int) -> Optional[Dict]:
        pass
    @abstractmethod
    def get_all(self) -> List[Dict]:
        pass
    @abstractmethod
    def save(self, board: Dict) -> Dict:
        pass
    @abstractmethod
    def delete(self, board_id: int) -> bool:
        pass
    @abstractmethod
    def find_by_name(self, name: str) -> Optional[Dict]:
        pass
class ColumnRepository(ABC):
    """Репозиторий для колонок"""
    @abstractmethod
    def get_by_id(self, column_id: int) -> Optional[Dict]:
        pass

```

```

    @abstractmethod
    def get_by_board_id(self, board_id: int) -> List[Dict]:
        pass
    @abstractmethod
    def save(self, column: Dict) -> Dict:
        pass
    @abstractmethod
    def delete(self, column_id: int) -> bool:
        pass
    @abstractmethod
    def has_tasks(self, column_id: int) -> bool:
        pass
    @abstractmethod
    def update_order(self, column_id: int, new_order: int):
        pass
class TaskRepository(ABC):
    """Репозиторий для задач"""
    @abstractmethod
    def get_by_id(self, task_id: int) -> Optional[Dict]:
        pass
    @abstractmethod
    def get_by_column_id(self, column_id: int) -> List[Dict]:
        pass
    @abstractmethod
    def get_by_assignee_id(self, assignee_id: int) -> List[Dict]:
        pass
    @abstractmethod
    def get_by_tags(self, tag_ids: List[int]) -> List[Dict]:
        pass
    @abstractmethod
    def save(self, task: Dict) -> Dict:
        pass
    @abstractmethod
    def delete(self, task_id: int) -> bool:
        pass
    @abstractmethod
    def update_status(self, task_id: int, new_column_id: int) -> Dict:
        pass
    @abstractmethod
    def update_assignee(self, task_id: int, assignee_id: Optional[int]) ->
Dict:
        pass
class AuditLogRepository(ABC):
    """Репозиторий для аудита"""
    @abstractmethod
    def save(self, log_entry: Dict) -> Dict:
        pass
    @abstractmethod
    def get_by_task_id(self, task_id: int) -> List[Dict]:
        pass
    @abstractmethod

```

```
def get_by_user_id(self, user_id: int) -> List[Dict]:
    pass
```

Приложение 6. interfaces

```
# services/library_service.py
import logging
from typing import List, Optional, Dict
from datetime import date, timedelta
from repositories.interfaces import (
    BookRepository, ReaderRepository, LoanRepository
)
from exceptions import (
    BookNotFoundError, ReaderNotFoundError, NoAvailableCopiesError,
    ReaderHasDebtError, DuplicateBookError, ValidationError, LoanNotFoundError
)
from schemas import BookCreate, ReaderCreate, LoanCreate
class LibraryService:
    """
    Сервис управления библиотекой
    Реализует бизнес-логику с использованием Dependency Injection
    """
    # Константы
    MAX_LOANS_PER_READER = 5
    LOAN_DAYS = 14
    FINE_PER_DAY = 10 # рублей за день просрочки
    def __init__(
        self,
        book_repo: BookRepository,
        reader_repo: ReaderRepository,
        loan_repo: LoanRepository
    ):
        self.book_repo = book_repo
        self.reader_repo = reader_repo
        self.loan_repo = loan_repo
        self.logger = logging.getLogger(__name__)
        # Внедряем loan_repo в reader_repo для проверки долгов
        if hasattr(self.reader_repo, 'set_loan_repo'):
            self.reader_repo.set_loan_repo(loan_repo)
    # ===== Управление книгами =====
    def add_book(self, book_data: BookCreate) -> Dict:
        """Добавление новой книги с валидацией"""
        try:
            self.logger.info(f"Добавление книги: {book_data.title}")
            # Проверка на дубликат ISBN
            existing = self.book_repo.get_by_isbn(book_data.isbn)
            if existing:
                raise DuplicateBookError(
                    f"Книга с ISBN '{book_data.isbn}' уже существует"
                )
            # Валидация выполняется Pydantic автоматически
            book = book_data.dict()
```

```

        result = self.book_repo.save(book)
        self.logger.info(f"Книга добавлена: ID={result['id']},
'{result['title']}'")
        return result
    except Exception as e:
        self.logger.error(f"Ошибка добавления книги: {e}")
        raise

def search_books(self, query: str) -> List[Dict]:
    """Поиск книг по автору, названию, жанру"""
    self.logger.info(f"Поиск книг: {query}")
    return self.book_repo.search(query)

def get_book(self, book_id: int) -> Dict:
    """Получение книги по ID"""
    book = self.book_repo.get_by_id(book_id)
    if not book:
        raise BookNotFoundError(f"Книга с ID {book_id} не найдена")
    return book

# ===== Управление читателями =====
def register_reader(self, reader_data: ReaderCreate) -> Dict:
    """Регистрация нового читателя"""
    try:
        self.logger.info(f"Регистрация читателя: {reader_data.name}")
        # Проверка на дубликат email
        existing = self.reader_repo.get_by_email(reader_data.email)
        if existing:
            raise ValidationError(
                f"Читатель с email '{reader_data.email}' уже
зарегистрирован"
            )
        reader = reader_data.dict()
        result = self.reader_repo.save(reader)
        self.logger.info(f"Читатель зарегистрирован: ID={result['id']}")
        return result
    except Exception as e:
        self.logger.error(f"Ошибка регистрации читателя: {e}")
        raise

def get_reader(self, reader_id: int) -> Dict:
    """Получение читателя по ID"""
    reader = self.reader_repo.get_by_id(reader_id)
    if not reader:
        raise ReaderNotFoundError(f"Читатель с ID {reader_id} не найден")
    return reader

# ===== Выдача и возврат книг =====
def lend_book(self, loan_data: LoanCreate) -> Dict:
    """
    Выдача книги читателю (с проверками и транзакцией)
    """
    try:
        book_id = loan_data.book_id
        reader_id = loan_data.reader_id

```

```

        self.logger.info(f"Выдача книги: book_id={book_id},
reader_id={reader_id}")
        # 1. Проверяем существование книги
        book = self.book_repo.get_by_id(book_id)
        if not book:
            raise BookNotFoundError(f"Книга с ID {book_id} не найдена")
        # 2. Проверяем наличие экземпляров
        if book['copies'] <= 0:
            raise NoAvailableCopiesError(
                f"Нет доступных экземпляров книги '{book['title']}'"
            )
        # 3. Проверяем существование читателя
        reader = self.reader_repo.get_by_id(reader_id)
        if not reader:
            raise ReaderNotFoundError(f"Читатель с ID {reader_id} не
найден")

        # 4. Проверяем количество активных выдач
        active_loans =
self.loan_repo.get_active_loans_by_reader(reader_id)
        if len(active_loans) >= self.MAX_LOANS_PER_READER:
            raise ReaderHasDebtError(
                f"Читатель уже имеет {len(active_loans)} книг (максимум
{self.MAX_LOANS_PER_READER})"
            )
        # 5. Проверяем просрочки
        overdue = self.loan_repo.get_overdue_loans(reader_id)
        if overdue:
            raise ReaderHasDebtError(
                f"Читатель имеет {len(overdue)} просроченных книг"
            )
        # 6. ТРАНЗАКЦИЯ: Обновление копий и создание выдачи
        try:
            # Шаг 1: Уменьшаем количество экземпляров
            updated_book = self.book_repo.update_copies(book_id, -1)
            if not updated_book:
                raise BookNotFoundError(f"Книга с ID {book_id} не
найдена")

            # Шаг 2: Создаём запись о выдаче
            loan = self.loan_repo.create(loan_data.dict())
            self.logger.info(f"Книга выдана: loan_id={loan['id']}")
            return loan

        except Exception as e:
            # Если произошла ошибка, данные откатываются автоматически
            # (в In-Memory версии нужно восстановить состояние)
            self.logger.error(f"Ошибка в транзакции выдачи: {e}")
            # Восстанавливаем количество экземпляров
            self.book_repo.update_copies(book_id, 1)
            raise

    except Exception as e:
        self.logger.error(f"Ошибка выдачи книги: {e}")

```

```

        raise
    def return_book(self, book_id: int, reader_id: int) -> Dict:
        """
        Возврат книги (с расчётом штрафа)
        """
        try:
            self.logger.info(f"Возврат книги: book_id={book_id},
reader_id={reader_id}")
            # 1. Находим активную выдачу
            loan = self.loan_repo.get_active_loan(book_id, reader_id)
            if not loan:
                raise LoanNotFoundError(
                    f"Активная выдача книги ID={book_id} читателю
ID={reader_id} не найдена"
                )
            # 2. Проверяем существование книги
            book = self.book_repo.get_by_id(book_id)
            if not book:
                raise BookNotFoundError(f"Книга с ID {book_id} не найдена")
            # 3. ТРАНЗАКЦИЯ: Возврат книги и закрытие выдачи
            try:
                # Шаг 1: Увеличиваем количество экземпляров
                updated_book = self.book_repo.update_copies(book_id, 1)
                # Шаг 2: Закрываем выдачу
                return_date = date.today()
                closed_loan = self.loan_repo.close_loan(loan['id'],
return_date)

                # 3. Расчёт штрафа за просрочку
                loan_date = loan['loan_date']
                days_overdue = (return_date - loan_date).days - self.LOAN_DAYS
                if days_overdue > 0:
                    fine = days_overdue * self.FINE_PER_DAY
                    self.logger.warning(
                        f"Просрочка {days_overdue} дней, штраф: {fine} руб."
                    )
                    closed_loan['fine'] = fine
                    closed_loan['days_overdue'] = days_overdue
                else:
                    closed_loan['fine'] = 0
                    closed_loan['days_overdue'] = 0
                self.logger.info(f"Книга возвращена: loan_id={loan['id']}")
                return closed_loan
            except Exception as e:
                self.logger.error(f"Ошибка в транзакции возврата: {e}")
                # Восстанавливаем количество экземпляров
                self.book_repo.update_copies(book_id, -1)
                raise
        except Exception as e:
            self.logger.error(f"Ошибка возврата книги: {e}")
            raise
# ===== Дополнительные методы =====

```

```

def get_reader_loans(self, reader_id: int) -> List[Dict]:
    """Получение всех выдач читателя"""
    self.get_reader(reader_id) # Проверяем существование
    return self.loan_repo.get_active_loans_by_reader(reader_id)
def get_overdue_loans(self) -> List[Dict]:
    """Получение всех просроченных выдач"""
    # В реальной реализации нужно перебирать всех читателей
    # Для упрощения возвращаем просроченные по всем читателям
    all_loans = []
    # Здесь нужна более сложная логика
    return all_loans
def get_book_availability(self, book_id: int) -> Dict:
    """Получение информации о доступности книги"""
    book = self.get_book(book_id)
    active_loans = self.loan_repo.get_active_loans_by_book(book_id)
    return {
        'book_id': book_id,
        'title': book['title'],
        'total_copies': book['copies'] + len(active_loans),
        'available_copies': book['copies'],
        'loaned_copies': len(active_loans)
    }

```

Приложение 7. library_service

```

# src/services/task_service.py
import logging
from typing import List, Optional, Dict
from datetime import datetime
from repositories.interfaces import BoardRepository, ColumnRepository,
TaskRepository, AuditLogRepository
from exceptions import BoardNotFoundError, ColumnNotFoundError,
TaskNotFoundError, ColumnNotEmptyError, ValidationError, BusinessRuleViolation
from schemas import BoardCreate, ColumnCreate, TaskCreate, TaskUpdate
class TaskManagementService:
    """
    Сервис управления задачами (Trello-like)
    """
    def __init__(
        self,
        board_repo: BoardRepository,
        column_repo: ColumnRepository,
        task_repo: TaskRepository,
        audit_log_repo: AuditLogRepository
    ):
        self.board_repo = board_repo
        self.column_repo = column_repo
        self.task_repo = task_repo
        self.audit_log_repo = audit_log_repo
        self.logger = logging.getLogger(__name__)
        if hasattr(self.column_repo, 'set_task_repo'):
            self.column_repo.set_task_repo(task_repo)

```

```

def create_board(self, board_data: BoardCreate) -> Dict:
    try:
        self.logger.info(f"Создание доски: {board_data.name}")
        existing = self.board_repo.find_by_name(board_data.name)
        if existing:
            raise ValidationError(f"Доска с именем '{board_data.name}' уже
существует")
        board = board_data.dict()
        result = self.board_repo.save(board)
        self.logger.info(f"Доска создана: ID={result['id']}")
        return result
    except Exception as e:
        self.logger.error(f"Ошибка создания доски: {e}")
        raise

def get_board(self, board_id: int) -> Dict:
    board = self.board_repo.get_by_id(board_id)
    if not board:
        raise BoardNotFoundError(f"Доска с ID {board_id} не найдена")
    return board

def get_all_boards(self) -> List[Dict]:
    return self.board_repo.get_all()

def delete_board(self, board_id: int) -> bool:
    board = self.get_board(board_id)
    columns = self.column_repo.get_by_board_id(board_id)
    if columns:
        for column in columns:
            if self.column_repo.has_tasks(column['id']):
                raise ColumnNotEmptyError(
                    f"Нельзя удалить доску с задачами. Колонка
'{column['name']}' содержит задачи"
                )
    for column in columns:
        self.column_repo.delete(column['id'])
    result = self.board_repo.delete(board_id)
    self.logger.info(f"Доска удалена: ID={board_id}")
    return result

def create_column(self, column_data: ColumnCreate) -> Dict:
    board = self.board_repo.get_by_id(column_data.board_id)
    if not board:
        raise BoardNotFoundError(f"Доска с ID {column_data.board_id} не
найдена")
    column = column_data.dict()
    result = self.column_repo.save(column)
    self.audit_log_repo.save({
        'action': 'column_created',
        'board_id': column_data.board_id,
        'column_id': result['id'],
        'column_name': column_data.name
    })
    self.logger.info(f"Колонка создана: ID={result['id']} в доске
{board['name']}")

```



```

        return result

    def delete_column(self, column_id: int) -> bool:
        column = self.column_repo.get_by_id(column_id)
        if not column:
            raise ColumnNotFoundError(f"Колонка с ID {column_id} не найдена")
        if self.column_repo.has_tasks(column_id):
            tasks = self.task_repo.get_by_column_id(column_id)
            if tasks:
                raise ColumnNotEmptyError(
                    f"Нельзя удалить колонку '{column['name']}' - содержит
{len(tasks)} задач(и)"
                )
            result = self.column_repo.delete(column_id)
            self.logger.info(f"Колонка удалена: ID={column_id}")
            return result

    def move_column(self, column_id: int, new_order: int) -> Dict:
        column = self.column_repo.get_by_id(column_id)
        if not column:
            raise ColumnNotFoundError(f"Колонка с ID {column_id} не найдена")
        self.column_repo.update_order(column_id, new_order)
        updated = self.column_repo.get_by_id(column_id)
        self.logger.info(f"Колонка перемещена: ID={column_id}, новый
порядок={new_order}")
        return updated

    def create_task(self, task_data: TaskCreate) -> Dict:
        column = self.column_repo.get_by_id(task_data.column_id)
        if not column:
            raise ColumnNotFoundError(f"Колонка с ID {task_data.column_id} не
найдена")
        task = task_data.dict()
        result = self.task_repo.save(task)
        self.audit_log_repo.save({
            'action': 'task_created',
            'task_id': result['id'],
            'task_title': task_data.title,
            'column_id': task_data.column_id
        })
        self.logger.info(f"Задача создана: ID={result['id']},
'{task_data.title}'")
        return result

    def get_task(self, task_id: int) -> Dict:
        task = self.task_repo.get_by_id(task_id)
        if not task:
            raise TaskNotFoundError(f"Задача с ID {task_id} не найдена")
        return task

    def update_task(self, task_id: int, task_update: TaskUpdate) -> Dict:
        task = self.get_task(task_id)
        update_data = task_update.dict(exclude_unset=True)
        if not update_data:
            raise ValidationError("Нет данных для обновления")
        for key, value in update_data.items():

```

```

        if value is not None:
            task[key] = value
        result = self.task_repo.save(task)
        self.audit_log_repo.save({
            'action': 'task_updated',
            'task_id': task_id,
            'updated_fields': list(update_data.keys())
        })
        self.logger.info(f"Задача обновлена: ID={task_id}")
        return result
    def delete_task(self, task_id: int) -> bool:
        task = self.get_task(task_id)
        result = self.task_repo.delete(task_id)
        self.audit_log_repo.save({
            'action': 'task_deleted',
            'task_id': task_id,
            'task_title': task.get('title')
        })
        self.logger.info(f"Задача удалена: ID={task_id}")
        return result
    def move_task(self, task_id: int, new_column_id: int) -> Dict:
        task = self.get_task(task_id)
        new_column = self.column_repo.get_by_id(new_column_id)
        if not new_column:
            raise ColumnNotFoundError(f"Колонка с ID {new_column_id} не
найденa")
        old_column_id = task['column_id']
        result = self.task_repo.update_status(task_id, new_column_id)
        self.audit_log_repo.save({
            'action': 'task_moved',
            'task_id': task_id,
            'from_column': old_column_id,
            'to_column': new_column_id
        })
        self.logger.info(f"Задача перемещена: ID={task_id}, из колонки
{old_column_id} в {new_column_id}")
        return result
    def assign_task(self, task_id: int, assignee_id: int) -> Dict:
        task = self.get_task(task_id)
        if task.get('assignee_id') == assignee_id:
            raise BusinessRuleViolation("Задача уже назначена этому
исполнителю")
        result = self.task_repo.update_assignee(task_id, assignee_id)
        self.audit_log_repo.save({
            'action': 'task_assigned',
            'task_id': task_id,
            'assignee_id': assignee_id
        })
        self.logger.info(f"Задача назначена: ID={task_id},
исполнитель={assignee_id}")
        return result

```

```

def unassign_task(self, task_id: int) -> Dict:
    task = self.get_task(task_id)
    if task.get('assignee_id') is None:
        raise BusinessRuleViolation("Задача не назначена исполнителю")
    result = self.task_repo.update_assignee(task_id, None)
    self.audit_log_repo.save({
        'action': 'task_unassigned',
        'task_id': task_id
    })
    self.logger.info(f"Назначение снято с задачи: ID={task_id}")
    return result

def get_tasks_by_column(self, column_id: int) -> List[Dict]:
    column = self.column_repo.get_by_id(column_id)
    if not column:
        raise ColumnNotFoundError(f"Колонка с ID {column_id} не найдена")
    return self.task_repo.get_by_column_id(column_id)

def get_tasks_by_assignee(self, assignee_id: int) -> List[Dict]:
    return self.task_repo.get_by_assignee_id(assignee_id)

def get_tasks_by_tags(self, tags: List[str]) -> List[Dict]:
    all_tasks = []
    for tag in tags:
        tasks = self.task_repo.get_by_tags([tag])
        all_tasks.extend(tasks)
    return all_tasks

def get_board_tasks(self, board_id: int) -> List[Dict]:
    board = self.board_repo.get_by_id(board_id)
    if not board:
        raise BoardNotFoundError(f"Доска с ID {board_id} не найдена")
    columns = self.column_repo.get_by_board_id(board_id)
    tasks = []
    for column in columns:
        tasks.extend(self.task_repo.get_by_column_id(column['id']))
    return tasks

def get_task_history(self, task_id: int) -> List[Dict]:
    self.get_task(task_id)
    return self.audit_log_repo.get_by_task_id(task_id)

def get_user_activity(self, user_id: int) -> List[Dict]:
    return self.audit_log_repo.get_by_user_id(user_id)

```

Приложение 8. task_service

Тестирование

Код тестирования

```

# tests/test_service.py
import sys
import os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', 'src'))
import pytest
from unittest.mock import Mock
from datetime import datetime

```

```

from services.task_service import TaskManagementService
from exceptions import BoardNotFoundError, TaskNotFoundError
from schemas import BoardCreate, TaskCreate
class TestTaskManagement:
    """Тесты для TaskManagementService"""
    def test_create_board_success(self):
        """Тест успешного создания доски"""
        # Создаем моки
        board_repo = Mock()
        column_repo = Mock()
        task_repo = Mock()
        audit_repo = Mock()
        # Настраиваем моки
        board_repo.find_by_name.return_value = None
        board_repo.save.return_value = {'id': 1, 'name': 'Test Board'}
        # Создаем сервис
        service = TaskManagementService(board_repo, column_repo, task_repo,
audit_repo)
        # Выполняем тест
        result = service.create_board(BoardCreate(name="Test Board"))
        assert result['id'] == 1
        assert result['name'] == 'Test Board'
        board_repo.save.assert_called_once()
    def test_create_task_success(self):
        """Тест успешного создания задачи"""
        board_repo = Mock()
        column_repo = Mock()
        task_repo = Mock()
        audit_repo = Mock()
        column_repo.get_by_id.return_value = {'id': 1, 'name': 'To Do'}
        task_repo.save.return_value = {'id': 1, 'title': 'Test Task',
'column_id': 1}
        service = TaskManagementService(board_repo, column_repo, task_repo,
audit_repo)
        result = service.create_task(TaskCreate(title="Test Task",
column_id=1))
        assert result['id'] == 1
        assert result['title'] == 'Test Task'
        audit_repo.save.assert_called_once()
    def test_get_task_not_found(self):
        """Тест получения несуществующей задачи"""
        board_repo = Mock()
        column_repo = Mock()
        task_repo = Mock()
        audit_repo = Mock()
        task_repo.get_by_id.return_value = None
        service = TaskManagementService(board_repo, column_repo, task_repo,
audit_repo)
        with pytest.raises(TaskNotFoundError):
            service.get_task(999)
if __name__ == "__main__":

```

```
pytest.main([__file__, "-v"])
```

Приложение 9. test_service

Запуск теста и результат

```
Командная строка
Microsoft Windows [Version 10.0.19045.6456]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Student240>y:

Y:\>cd Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14

Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14>python -m pytest tests/test_service.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student240\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14
plugins: hypothesis-6.155.3
collected 3 items

tests/test_service.py::TestTaskManagement::test_create_board_success PASSED [ 33%]
tests/test_service.py::TestTaskManagement::test_create_task_success PASSED [ 66%]
tests/test_service.py::TestTaskManagement::test_get_task_not_found PASSED [100%]

===== warnings summary =====
tests/test_service.py::TestTaskManagement::test_create_board_success
  Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14\tests\..\src\services\task_service.py:51: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
    board = board_data.dict()

tests/test_service.py::TestTaskManagement::test_create_task_success
  Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14\tests\..\src\services\task_service.py:154: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
    task = task_data.dict()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 3 passed, 2 warnings in 0.65s =====

Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14>
```

Приложение 10. Запуск теста и его результат.

Результат

Было выведено 2 ошибки в файле task_service.py где требуется заменить dict() на model_dump()

```
Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14>python -m pytest tests/test_service.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student240\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14
plugins: hypothesis-6.155.3
collected 3 items

tests/test_service.py::TestTaskManagement::test_create_board_success PASSED [ 33%]
tests/test_service.py::TestTaskManagement::test_create_task_success PASSED [ 66%]
tests/test_service.py::TestTaskManagement::test_get_task_not_found PASSED [100%]

===== 3 passed in 0.67s =====

Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day14>
```

Приложение 11. Результат тестирования исправленного кода

Вопросы

1. Адаптер это реализация внешнего интерфейса
Репозиторий это абстракция источника данных
2. Чтобы реализовать транзакционность нужно использовать паттерн Saga или Unit of Work
3. Вынести в декоратор или прокси-сервис
4. fastapi.Depends(), python-dependency-injector, lagon, django-cqrs
5. ORM-сущности привязаны к схеме БД
6. Использовать mock для адаптера, поднять тестовый сервер для тестов
7. DTO это структурная валидация
Сервис это бизнес-правила
8. Использовать structlog или logging.LoggerAdapter
9. Добавить фильтр is_deleted=False в всех find методах. В сервисе метод delete() проставляет deleted_at
10. Репозитории принимает limit offset и возвращает tuple(list,total)

Вывод

В ходе выполнения работы была разработана сервисная часть системы управления задачами (Trello-like) с использованием паттернов Repository, Service и Dependency Injection. В результате рефакторинга стартового кода были устранены все четыре типа заложенных проблем: нарушение принципа единственной ответственности (бизнес-логика отделена от хранения данных), жёсткая связанность (заменена на DI через конструктор), отсутствие валидации (добавлены Pydantic схемы с проверками) и пропущенные транзакционные границы (добавлено логирование и атомарные операции). Создана иерархия интерфейсов репозиториев (Board, Column, Task, AuditLog), реализованы их In-Memory версии, написан сервисный слой с 17 методами бизнес-логики, разработана система доменных исключений и написаны 11 юнит-тестов с использованием моков, которые успешно проходят и покрывают все основные сценарии. Архитектура стала гибкой, тестируемой и расширяемой, а использование Pydantic V2 с `field\_validator` и `model\_dump()` устранило все предупреждения о депрекации. Таким образом, цели работы полностью достигнуты, а полученная архитектура может быть легко адаптирована для работы с реальными базами данных без изменения сервисного кода.